AURA / MISSION 01

The Architecture of Argument & Dialogic Protocols

Squadron Fei-Fei | Conversational AI | Winter Semester 2026/27

IMPLEMENTATION REPORT - verified live Gemini execution

Members: Bilal Younus, Danish Hafeez and Anas Arshad.

Team roles

Three team members cover the four mission roles.

Member	Assigned responsibilities
Bilal Younus	Core Architect: implementation, run evidence and integration
Danish Hafeez	Protocol & Interface Specialist: state map, pseudocode and termination
Anas Arshad	Knowledge & Memory Archivist; Security & Ethical Navigator: philosophy, evaluation, personas and safety

Three members cover four station roles. Bilal also coordinates GitHub setup and submission.

Evidence status

The integrated engine completed three Socratic turns and a four-turn Galilean debate using Gemini 3.1 Flash-Lite. The full transcript and exact API prompts/outputs accompany this report. Ten offline software tests also pass. A qualitative evaluation identifies both useful reasoning and substantive weaknesses in the generated dialogue.

Execution evidence documents a Python terminal run.

01 / Dialogic alignment brief

Discovery through contradiction

In Plato's *Euthyphro*, Socrates asks for a general definition of piety rather than a list of pious actions. Successive definitions are tested against their implications. The question of whether the pious is loved by the gods because it is pious, or is pious because it is loved, exposes a distinction between a property and an explanation of that property. The dialogue ends without a settled definition: aporia can be an honest outcome of inquiry.

A computational analogue starts with a premise P0, clarifies its terms, generates a counterexample and identifies the precise conflict. It then refines the premise and tests again. The system should not equate failure to find a contradiction with proof. A finite iteration budget and explicit unresolved status prevent an endless or falsely conclusive exchange.

Persuasion through contrasting voices

Galileo's *Dialogue Concerning the Two Chief World Systems* uses Salviati, Simplicio and Sagredo to develop competing arguments. In this mission, the characters map to Advocate, Traditionalist and Mediator. Their distinct responsibilities make assumptions and objections visible. The historical work is a crafted argument, however, not a neutral computational experiment; persona diversity alone does not establish truth.

Aspect	Socratic discovery	Galilean debate
Progress	A more precise definition	A better-supported assessment of trade-offs
Mechanism	Probe, counterexample, refine	Argument, critique, rebuttal, synthesis
Main risk	Leading questions; false resolution	Shared bias; theatrical disagreement
Control	Aporia and finite testing	Evidence labels and independent mediation

AURA discovery persona

Clarify claims rather than flatter the user. Identify ambiguous terms, ask focused questions, and explain exactly how a counterexample tests the premise. Revise only when warranted. Separate facts, assumptions and values. Do not invent evidence. Acknowledge aporia when a key uncertainty cannot be resolved.

Modern-model evaluation must examine probing quality, logical testing, persona distinction, evidence discipline and resistance to uncritical agreement. Our live Gemini case study evaluates these properties in the integrated dialogue. It supports protocol-specific observations, not a general ranking of Gemini, ChatGPT and Claude.

02 / Socratic protocol

Input: a non-empty premise. Example: "An AI agent is benevolent if it obeys human commands."

State	Action / next state
Receive	Validate premise; set iteration count and history.
Probe and test	Clarify terms, ask questions, generate counterexample and assess conflict.
Validate response	Require QUESTIONS, COUNTEREXAMPLE, TEST, REFINED PREMISE and a valid STATUS. Malformed output fails the run.
REFINE	If premise changes, update it and continue with history. No meaningful change leads to APORIA.
RESOLVED_WITH_CAVEATS	Return the tested premise; this is not universal proof.
APORIA / iteration limit	Return an unresolved status and the best current premise.

Worked conceptual example

An instruction to disclose another person's private information is a counterexample to equating obedience with benevolence. A refinement might require legitimate instructions plus privacy and safety constraints. The next probe asks who decides legitimacy and how conflicting obligations are handled. Without those governance criteria, the engine should preserve uncertainty.

Implementation and limits

The Python controller permits at most three Socratic iterations. It supplies earlier responses as context and validates required output sections and terminal status. Parsing is deliberately simple and fails on an unexpected format instead of silently treating it as success. The model still supplies the substantive judgment about contradictions: a valid format does not guarantee valid reasoning.

The schematic below matches the implemented transition logic:

Premise → **probe/test** → **validate** → **status**
REFINE → changed premise? → yes: next iteration; no: APORIA
RESOLVED_WITH_CAVEATS → terminate with qualification
APORIA or exhausted budget → terminate unresolved

03 / Galilean ethics sandbox

Thought experiment: a consequential release

A university laboratory has developed a model useful for education and programming. Researchers want public weights and code to enable independent scrutiny; smaller organisations want affordable access. Capability testing is incomplete, and the security team raises unresolved cyber-misuse concerns. Once copied, the weights cannot reliably be recalled. The decision is whether to release immediately, permit staged evaluation access, or restrict access.

Persona	Responsibility
Salviati - Advocate	Present the strongest reasonable case for openness, including research, reproducibility and access; state assumptions.
Simplicio - Traditionalist	Challenge the case through misuse, privacy, irreversibility and missing evidence; avoid assuming all releases are unsafe.
Sagredo - Mediator	Compare evidence and trade-offs, explain a conditional recommendation and preserve unresolved disagreements.

Execution order: Advocate opening → Critic response → Advocate rebuttal → Mediator synthesis. The mediator receives all prior arguments, including the rebuttal. Each persona is a separate API call to the same configured model, not an independent model or source of evidence.

Safety audit

Sycophancy: a dedicated critic must challenge the proposal. The advocate must respond to the strongest objection rather than repeat the opening. This may reveal weak assumptions but does not eliminate agreement bias.

Correlated errors: all personas share the same model. Independent evidence review and human assessment remain necessary. Three agreeing personas are not three independent confirmations.

False balance: the mediator should weight evidence rather than average viewpoints. When evidence is incomplete, recommendations must be conditional rather than confidently definitive.

Unsupported claims: require uncertainty labels, record the full output and verify factual assertions. This implementation does not automatically check external sources or quantify actual model-release risk.

Decision authority: the debate is analytical support. It does not authorize publication of a model or substitute for accountable human governance.

04 / Engine verification and evaluation

Changes verified offline

The revised implementation converts exhausted refinement loops to aporia, detects unchanged premises, validates model output labels and statuses, retains previous Socratic turns, rejects empty or truncated API replies, adds an advocate rebuttal and records prompts plus returned model metadata. Setup or execution errors return a nonzero process status.

Test coverage	Result
Iteration exhaustion and unchanged premise	Passed
Explicit aporia and qualified resolution	Passed
Invalid status and missing sections	Passed
Conversation history and rebuttal propagation	Passed
Empty input and empty/truncated API response	Passed

Total: 10 tests passed. Tests use controlled fixture replies and exercise program behavior. They are not evidence of model reasoning quality or successful API authentication.

Empirical model evaluation

Gemini 3.1 Flash-Lite ran at temperature 0.4 with low thinking level and a requested 180-word response limit. The run was resumed after network interruptions using cached genuine API responses. Exact prompts, timestamps and outputs are preserved. This is a single-model qualitative case study, reviewed with AI assistance for team verification; it is not a cross-provider benchmark.

Discovery: three turns progress from obedience to well-being, then pluralistic governance, and finally rights protection and oversight. The model challenges each emerging definition. Its qualified resolution still leaves the meaning of fundamental rights and inclusive oversight unresolved.

Debate: the critic raises leakage and red-teaming limitations; the advocate adds stopping thresholds in response. The mediator acknowledges speculative evidence and recommends conditional staged access. However, the advocate moves quickly to compromise, limiting viewpoint diversity.

Critical finding: the mediator claims API-only access prevents weight exfiltration, then asks whether it reliably does so. API-only access does not guarantee infrastructure security. This overstatement illustrates shared-model error propagation and the need for human review. The raw transcript is retained unchanged.

OpenAI generation was unavailable because account credit was exhausted; Claude was not tested. No comparative performance scores are inferred. The detailed evaluation is in 07_Model_Evaluation.md.

05 / Reproduction and submission

Run locally

Install requirements, run **python3 run_securely.py** and enter GEMINI_API_KEY at the hidden prompt. Press Enter for the default premise and debate scenario. Set AURA_MODEL=gemini-3.1-flash-lite to reproduce the verified model. Cached genuine responses allow interrupted runs to resume; remove run_cache for a fresh sample. Successful completion writes 05_Execution_Transcript.txt and 06_API_Run_Evidence.json next to the script. Review both files before using them as evidence.

Run **python3 -m unittest discover -p test_engine.py** to reproduce the offline tests. Open the project in the course-required VS Code/Antigravity workflow and document what was actually used.

Submission checklist

1. Review the complete transcript and empirical evaluation.
2. Confirm station sign-offs and record the course-required IDE workflow.
3. Publish the reviewed repository.
4. Submit this report and the public repository URL through the course portal.

The pasted Moodle page specifies 20 October 2026, 23:00 Europe/Berlin and accepts PDFs/URLs. The assignment also mentions the AURA console and a repository relationship to the course repository. Verify the intended submission route and repository arrangement; this package does not claim submission or publication.

Sources and provenance

Assignment source: ps01.pdf, Mission 1: The Architecture of Argument & Dialogic Protocols, supplied by the user. Starting materials: AURA_Mission1_Assignment.zip, supplied by the user. Administrative context: pasted Moodle assignment and course announcement.

Philosophical works discussed in the assignment: Plato, Euthyphro (definition of piety and the explanatory dilemma); Galileo Galilei, Dialogue Concerning the Two Chief World Systems (three-character argumentative format). No verbatim passages or edition-specific page claims are used.

API implementation reference: OpenAI, Structured model outputs. The engine validates a plain-text label contract locally. Gemini generation uses its native REST endpoint; OpenAI is an optional provider.

Tools: Codex assisted with code revision, tests and report preparation; Gemini generated the recorded dialogue.